

혼자 공부하는 바이브코딩 · Study-01 ~ Study-06 복습

바이브코딩 개념 정리 노트

캐시와 쿠키 같은 웹의 기초부터
스킬 · MCP · 플러그인까지, 개념끼리 어떻게 이어지는지

읽는 시간: 약 2시간 · 코딩 없이 읽기만 해도 되도록 썼어요
예시는 모두 우리가 직접 한 실습(Study-01~06)에서 가져왔어요

이 노트를 읽는 법

이 노트는 **외우는 자료가 아니라 "아, 그게 그거였구나" 하고 연결하는 자료**예요. 처음부터 순서대로 읽는 걸 추천하지만, 궁금한 부분부터 봐도 괜찮아요. 모든 개념에는 네 가지 상자가 붙어 있어요.

비유

일상에 빗대어 감을 잡는 부분이에요.

우리 실습에서

그 개념이 우리 Study 폴더 어디에 나왔는지 보여 줘요.

기억할 것

딱 하나만 가져가면 되는 핵심이에요.

회색 줄이 붙은 문단은 "지금은 몰라도 되는 부분"이에요. 가볍게 넘겨도 괜찮아요. 진행하다 보면 자연스럽게 익숙해질 부분이에요.

목차

0장. 큰 그림 한 장

1부. 웹의 기초 — 인터넷에서 페이지 하나가 뜨기까지

클라이언트와 서버 · URL · HTTP와 상태 코드 · HTML/CSS/JS · DOM과 이벤트 · JSON · 쿠키와 세션 · localStorage · 출처 (Origin) · 캐시 · 보안 기초 · 네트워크와 프록시

2부. 개발 도구의 기초 — 코드를 만들고 보관하고 확인하는 도구들

파일과 경로 · 터미널 · 언어와 런타임 · 패키지 · Git과 GitHub · 로컬·클라우드·컨테이너 · 테스트 · 오류 읽는 법

3부. AI의 기초 — LLM이 무엇이고 어떻게 불러 쓰는가

AI·머신러닝·LLM · 토큰과 컨텍스트 · 추론·멀티모달 · API로 AI 쓰기 · 에이전트

4부. Claude Code 확장 체계 — 이 노트의 핵심

CLAUDE.md · 커스텀 명령어 · 스킬 · 서브에이전트 · MCP · 훅 · 설정과 권한 · 플러그인과 마켓플레이스 · 한눈 비교표 · 언제 무엇을 쓰나

5부. 실습 되짚기 — Study-01부터 06까지, 그리고 오늘 있었던 일

6부. 스스로 점검하기 — 20문제와 해설

부록. 용어 사전 — 한 줄 정의 모음

0장. 큰 그림 한 장

세부 개념에 들어가기 전에, 우리가 공부한 것들이 어디쯤에 있는지 지도부터 보고 갈게요.

우리가 배운 것은 네 층으로 쌓여 있어요

4층 · Claude Code 확장 — CLAUDE.md, 커스텀 명령어, 스킴, 서브에이전트, MCP, 혹은 플러그인, 마켓플레이스
→ "AI 개발 도우미를 내 입맛에 맞게 길들이는 방법"

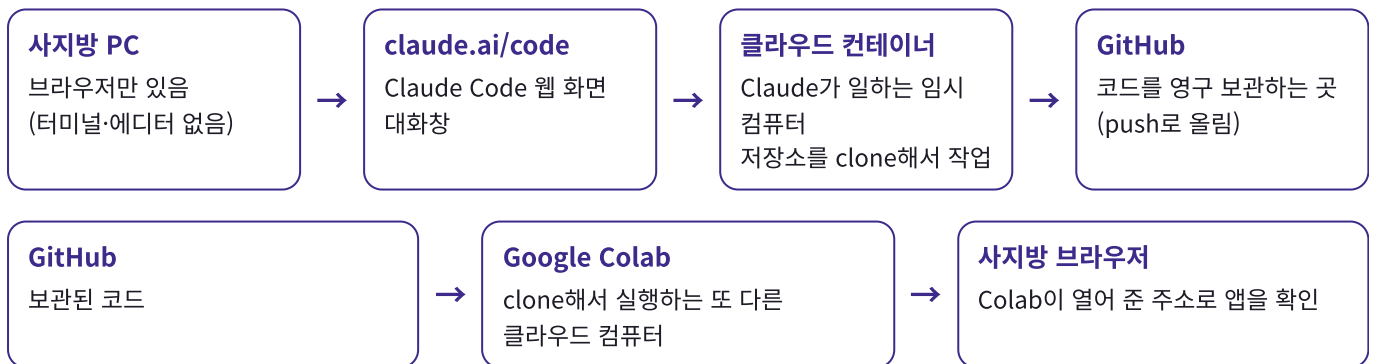
3층 · AI — LLM, 토큰, 프롬프트, API 키, OpenRouter, 에이전트
→ "AI가 무엇이고, 어떻게 불러 쓰는가"

2층 · 개발 도구 — 터미널, Git/GitHub, 패키지(pip, npm), 클라우드·컨테이너, 테스트
→ "코드를 만들고, 보관하고, 확인하는 도구"

1층 · 웹의 기초 — 서버와 클라이언트, URL, HTTP, HTML/CSS/JS, 쿠키, 캐시, localStorage
→ "우리가 만든 앱들이 돌아가는 바탕"

아래층이 위층을 받쳐요. 예를 들어 4층의 **MCP**로 Playwright를 연결할 때, 실제로는 1층의 **localhost와 포트**(앱을 띄운 주소), 2층의 **npm 패키지**(Playwright 설치), 3층의 **에이전트**(Claude가 도구를 골라 쓰는 방식)가 모두 함께 움직였어요. 그래서 한 층만 알면 늘 반쯤 이해한 느낌이 들고, 층끼리 연결되는 순간 "아!" 하게 돼요.

우리가 일하는 환경은 이렇게 생겼어요



📌 기억할 것

우리는 "내 컴퓨터"에서 일하지 않아요. **Claude**는 **클라우드 컨테이너**에서, 나는 **Colab**에서 각자 다른 컴퓨터로 일하고, 두 곳을 이어 주는 다리가 **GitHub**예요. 그래서 "커밋하고 push했어요"가 중요한 거예요. push하지 않으면 Colab에서는 그 코드를 볼 수 없어요.

1부. 웹의 기초

할 일 앱(Study-02), 퀴즈(Study-03), 근무표(Study-05), 쇼핑 리스트(Study-06)는 모두 웹 앱이에요. 웹 페이지 하나가 화면에 뜨기까지 무슨 일이 일어나는지부터 볼게요.

1-1 클라이언트와 서버

서버(server)는 요청을 받아서 무언가를 내주는 컴퓨터(또는 프로그램)이고, **클라이언트(client)**는 요청을 보내는 쪽이에요. 웹에서는 대개 **브라우저(크롬, 엣지)**가 **클라이언트**, 웹사이트를 가진 컴퓨터가 서버예요.

비유 — 식당

손님(클라이언트)이 주문서를 내면(요청), 주방(서버)이 음식을 만들어 내줘요(응답). 주문서를 쓰는 형식이 정해져 있어야 주방이 알아들겠죠? 그 형식이 바로 뒤에 나올 **HTTP**예요.

중요한 점은 "**서버**"는 특별한 기계가 아니라 **역할**이라는 거예요. 평범한 노트북도 서버 프로그램을 켜면 서버가 돼요.

우리 실습에서

- Study-05 근무표: `timetable_server/` 가 서버예요. 브라우저가 "AI로 선호도 해석해 줘"라고 요청하면 서버가 OpenRouter에 물어보고 결과를 돌려줘요.
- Study-06: 쇼핑 리스트를 테스트할 때 `python3 -m http.server 8765` 로 **1초 만에 서버를 만들었어요**. "이 폴더의 파일을 달라고 하면 내줘라"만 하는 아주 단순한 서버예요.
- Study-04: Streamlit 앱도 실행하면 작은 서버가 켜지고, 브라우저가 거기 접속해요.

반대로 쇼핑 리스트 앱 자체는 **서버가 필요 없는 앱**이에요. HTML 파일 하나에 모든 것이 들어 있고, 데이터도 브라우저 안(localStorage)에 저장하니까요. 그래서 더블클릭만으로 열려요. 이런 앱을 "정적(static) 페이지" 또는 "클라이언트만 있는 앱"이라고 불러요.

1-2 URL — 인터넷 주소 읽는 법

URL은 "어느 컴퓨터의, 어떤 프로그램에게, 무엇을 달라고 할지"를 한 줄로 쓴 주소예요. 오늘 실제로 쓴 주소를 뜯어볼게요.

https	://	vibeindex.ai	:443	/api/stats	?ref=skill-vibeindex
프로토콜		도메인	포트	경로	쿼리

부분	뜻	비유
프로토콜 <code>https</code>	어떤 약속(규칙)으로 대화할지. <code>http</code> 는 그냥, <code>https</code> 는 암호화해서 대화해요. <code>file</code> 은 "인터넷 말고 이 컴퓨터의 파일을 바로 열어라"예요.	편지를 보통우편으로 보낼지, 봉인된 등기로 보낼지

도메인 vibeindex.ai	어느 컴퓨터인지. 사람이 읽기 쉬운 이름이에요.	건물 이름
포트 :443	그 컴퓨터 안의 어느 프로그램(창구)인지. https는 443, http는 80이 기본이라 보통 생략돼요.	건물 안 호실 번호
경로 /api/stats	그 프로그램 안에서 무엇을 원하는지	호실 안의 어느 서랍
쿼리 ?ref=...	추가 조건. 이름=값 을 & 로 이어 붙여요.	주문서의 옵션란("곱빼기")

localhost와 포트 번호

localhost(= 127.0.0.1)는 "바로 이 컴퓨터 자신"을 가리키는 특별한 이름이에요. `http://localhost:8765/`는 "내 컴퓨터의 8765번 창구에서 돌고 있는 프로그램"이라는 뜻이에요.

🔥 우리 실습에서 나온 포트들

- 8765 : 쇼핑 리스트 테스트용으로 띄운 서버
- 8000 : Colab에서 쇼핑 리스트를 열 때 안내한 서버
- 8501 : Streamlit의 기본 포트 (Study-04 냉장고 레시피)
- 443 : https 사이트 (vibeindex.ai, github.com 등)

포트 번호는 0~65535 중 아무거나 쓸 수 있지만, 이미 다른 프로그램이 쓰고 있으면 겹쳐서 실패해요. 그래서 8765처럼 잘 안 쓰는 번호를 골랐어요.

Colab에서 `serve_kernel_port_as_window(8000)` 을 쓰는 이유도 여기 있어요. Colab 컴퓨터의 localhost:8000은 **Colab 컴퓨터 자신**이라 사지방 PC에서는 접속할 수 없어요. 그래서 Colab이 "밖에서도 들어올 수 있는 임시 주소"를 만들어 연결해 주는 거예요. 이것 **포트 포워딩(port forwarding)**이라고 해요.

file:// 주소는 왜 막혔나

Playwright로 쇼핑 리스트를 열 때 `file:///home/user/.../index.html` 은 "보안상 막혀 있다"는 오류가 났어요. `file://` 로 아무 파일이나 열 수 있으면, 나쁜 웹페이지가 시를 속여 비밀 파일을 읽게 만들 수도 있거든요. 그래서 Playwright는 기본으로 막아 두고, 우리는 파일을 모두 여는 옵션 대신 **작은 서버를 띄워 http://localhost로 여는 더 안전한 길**을 택했어요.

1-3 HTTP — 요청과 응답

HTTP는 브라우저와 서버가 주고받는 **주문서와 영수증의 양식**이에요. 요청 하나에는 이런 게 들어 있어요.

- **메서드**: 무엇을 하고 싶은지. `GET` (가져와 줘), `POST` (이걸 받아서 처리해 줘)가 제일 흔해요. OpenRouter에 질문을 보낼 때는 `POST`를 썼어요(질문 내용을 담아 보내야 하니까).
- **헤더**: 부가 정보. 예) "나는 한국어를 원해", "내 API 키는 이거야", "내가 가진 쿠키는 이거야".
- **본문(body)**: 실제 내용. `POST`일 때 질문 내용(JSON) 같은 게 들어가요.

응답에는 **상태 코드**라는 세 자리 숫자가 붙어요. 이 숫자만 봐도 무슨 일이 있었는지 대충 알 수 있어서, 오류를 읽을 때 정말 유용해요.

코드	뜻	우리가 실제로 만난 경우
200 OK	성공	vibeindex.ai 접속 허용 후 추천 목록을 받았을 때
3xx	"다른 곳으로 가세요"(이동)	—
401	누군지 모르겠음(인증 실패). API 키가 없거나 틀림	Study-04·05에서 대비한 "키를 확인해 주세요" 메시지
403	누군지는 알지만 허락 안 됨(금지)	컨테이너에서 vibeindex.ai와 GitHub API에 접속하려다 막혔을 때
404	그런 것 없음	쇼핑 리스트 테스트 중 favicon.ico (탭 아이콘)가 없다는 콘솔 오류
429	너무 자주 요청함, 잠시 쉬세요	무료 AI 모델을 연달아 부를 때 대비한 메시지
500·502·503	서버 쪽 문제	AI 서비스가 잠시 불안정할 때

📌 기억할 것

4로 시작하면 "요청한 쪽(나)"의 문제, 5로 시작하면 "서버 쪽" 문제예요. 401은 키 확인, 403은 권한·차단 확인, 404는 주소 오타 확인, 429는 잠깐 기다리기. 이것만 알아도 오류의 절반은 읽혀요.

1-4 HTML · CSS · JavaScript

웹 페이지는 세 가지 언어가 역할을 나눠 맡아요.

언어	역할	집에 비유하면	쇼핑 리스트에서
HTML	무엇이 있는지(구조, 내용)	뼈대와 벽, 방	제목, 입력칸, [추가] 버튼, 목록 틀
CSS	어떻게 보이는지(모양)	벽지, 가구 배치, 조명	둥근 모서리, 파란 버튼, 체크하면 줄 굵기
JavaScript(JS)	어떻게 움직이는지(동작)	전기와 배선 — 스위치를 누르면 불이 켜짐	버튼을 누르면 목록에 추가, 저장, 개수 세기

쇼핑 리스트는 이 세 가지를 **파일 하나(index.html)**에 다 넣었어요. `<style>` 안이 CSS, `<script>` 안이 JS예요. Study-02 할 일 앱은 index.html, style.css, app.js 로 나눠 뒀죠. 어느 쪽이든 브라우저에게는 똑같아요. 나누면 큰 앱을 관리하기 쉽고, 합치면 파일 하나만 옮기면 되니 간편해요.

반응형(responsive) 디자인

CSS의 `@media (max-width: 520px) { ... }` 같은 부분은 "화면 폭이 520px 이하일 때만 이 모양을 써라"라는 뜻이에요. 휴대폰과 PC에서 알맞게 모양이 바뀌는 걸 반응형이라고 해요. Study-02의 web_version 이 PC용 2단 배치를 추가한 것도 이 방법이에요.

1-5 DOM과 이벤트, 그리고 콘솔

브라우저는 HTML을 읽고 나서 그걸 **나무 모양의 목록(DOM, Document Object Model)**으로 바꿔 기억해요. JavaScript는 이 DOM을 보고 고치는 방식으로 화면을 바꿔요.

```
문서
└─ body
   └─ main
      ├── h1 "🛒 쇼핑 리스트"
      ├── form (입력칸 + 추가 버튼)
      ├── p "전체 3개 · 체크 1개 ..."
      └─ ul (목록)
         ├── li [체크칸] "우유" [삭제]
         └─ li [체크칸] "사과" [삭제]
```

이벤트(event)는 "무슨 일이 일어났다"는 신호예요. 클릭(click), 입력칸 제출(submit), 체크 변경(change), 키 누름(keydown) 같은 거요. JS는 "이 버튼에서 click 이벤트가 생기면 이 함수를 실행해"라고 미리 걸어 뒀요(이벤트 리스너).

🔗 쇼핑 리스트의 설계

쇼핑 리스트 코드는 두 부분으로 나뉘어 있어요.

1. **데이터 처리 함수** (`addItem` , `removeItem` , `toggleItem`): 목록을 받아서 새 목록을 돌려줄 뿐, 화면은 건드리지 않아요.
2. **화면 그리기** (`render`): 목록을 보고 DOM을 새로 그려요.

Study-05 근무표 규칙("계산 로직은 DOM을 건드리지 않는 함수로")도 같은 생각이에요. 이렇게 나누면 계산이 맞는지 화면 없이도 테스트할 수 있고, 화면을 바꿔도 계산은 그대로 둘 수 있어요.

개발자 도구와 콘솔

크롬에서 **F12**를 누르면 개발자 도구가 열려요. 그중 **콘솔(Console)** 탭에는 JS가 남긴 메시지와 빨간 오류가 보여요. "버튼을 눌러도 아무 일이 없다"면 제일 먼저 콘솔을 보면 돼요. Study-05의 `runTests()` 를 실행하는 곳도 이 콘솔이었어요. Playwright 테스트에서 "콘솔 오류 없음"을 확인한 것도 같은 곳을 자동으로 본 거예요.

1-6 JSON — 데이터를 글자로 적는 약속

JSON(제이슨)은 데이터를 **사람도 읽고 컴퓨터도 읽을 수 있는 글자로 적는 형식**이에요. 쇼핑 리스트는 목록을 이렇게 저장해요.

```
[
  { "id": 1727..., "name": "우유", "done": true },
  { "id": 1727..., "name": "사과", "done": false }
]
```

- `[ ]` 는 목록(배열), `{ }` 는 "이름: 값" 묶음(객체)이에요.

- JS에서 객체를 JSON 글자로 바꾸는 게 `JSON.stringify`, 글자를 다시 객체로 바꾸는 게 `JSON.parse` 예요.
- 실패 하나만 빠져도 "깨진 JSON"이 되어 읽을 수 없어요. 그래서 쇼핑 리스트는 불러오기를 `try ... catch` 로 감싸서, 깨져 있으면 멈추지 않고 빈 목록으로 시작하게 했어요. 오늘 28개 테스트 중 "저장 값이 깨져도 멈추지 않음"이 이걸 확인한 거예요.

🌸 JSON은 어디에나 있었어요

AI에게 보내는 질문과 받는 답(Study-04-05), 레시피 결과(Study-04 PRD 2단계), 프로필 저장 파일 `profiles.json`, 할 일 앱의 내보내기/가져오기, 그리고 Claude Code 설정 파일인 `.mcp.json` 과 `settings.json` 까지 모두 JSON이에요.

1-7 쿠키(Cookie)와 세션(Session)

HTTP에는 중요한 성질이 하나 있어요. **서버는 기본적으로 기억력이 없어요.** 요청 하나하나를 처음 보는 손님처럼 대해요. 그러면 로그인은 어떻게 유지될까요? 여기서 쿠키가 등장해요.

쿠키는 **서버가 브라우저에게 맡겨 두는 작은 메모**예요.

1. 로그인에 성공하면 서버가 응답에 "이 메모를 가지고 있어: `손님번호=a8f3...`"라고 적어 보내요(`Set-Cookie` 헤더).
2. 브라우저는 그 메모를 그 사이트 이름으로 저장해 뒹요.
3. 그 뒤로 같은 사이트에 요청할 때마다 **브라우저가 알아서 메모를 같이 보내요.**
4. 서버는 메모를 보고 "아, 아까 로그인한 그분이구나" 하고 알아봐요.

🍲 비유 — 목욕탕 사물함 열쇠

카운터(서버)는 손님 얼굴을 다 기억하지 못해요. 대신 들어올 때 번호가 적힌 열쇠(쿠키)를 줘요. 손님은 그 열쇠를 손목에 차고 다니고, 매점에서 뭘 사면 열쇠 번호로 계산해요. 카운터 장부에는 "15번 열쇠 = 이 손님, 음료 2개"라고 적혀 있어요. 이 **카운터 쪽 장부가 세션**이고, **손님 손목의 열쇠가 쿠키**예요.

- **세션:** 서버가 "이 손님은 누구고 지금까지 뭘 했는지"를 서버 쪽에 기억해 두는 것. 쿠키에는 보통 세션을 찾는 번호 (세션 ID)만 들어 있어요.
- **만료:** 쿠키에는 유효기간을 붙일 수 있어요. "로그인 상태 유지"를 체크하면 유효기간이 긴 쿠키를 주는 거예요.
- **크기:** 하나에 약 4KB로 작아요. 매 요청마다 따라가니 크면 느려지거든요.
- **추적 쿠키:** 광고 회사가 여러 사이트에 걸쳐 "같은 사람"임을 알아보려고 쓰는 쿠키예요. 웹사이트의 "쿠키 사용에 동의하시겠어요?" 배너는 주로 이것 때문에 생겼어요.

⚠ 쿠키가 곧 로그인이에요

로그인 쿠키를 남이 가져가면 비밀번호 없이도 내 계정으로 들어올 수 있어요. 그래서 공용 PC(사지방 포함)에서는 쓰고 나서 꼭 **로그아웃**하세요. 로그아웃하면 서버가 그 세션을 지우기 때문에 쿠키가 쓸모없어져요. 시크릿 창(새 InPrivate 창)은 창을 닫을 때 쿠키를 지워 줘서 공용 PC에서 쓰기 좋아요.

1-8 localStorage와 브라우저 저장소들

localStorage는 웹 페이지가 브라우저 안에 데이터를 저장하는 공간이에요. 쇼핑 리스트, 할 일 앱, 퀴즈 리더보드, 근무표가 모두 이걸 써서 "새로고침해도 남아 있게" 했어요.

	쿠키	localStorage	sessionStorage
누가 주로 쓰나	서버 (로그인 유지)	페이지의 JS (앱 데이터)	페이지의 JS (잠깐 쓰는 값)
서버로 자동 전송?	예, 매 요청마다	아니요	아니요
얼마나 오래	정한 유효기간까지	지울 때까지 계속	탭을 닫을 때까지
크기	약 4KB	보통 약 5MB	보통 약 5MB
우리 예	claude.ai·GitHub 로그인	쇼핑 리스트, 할 일, 퀴즈 점수	—

더 큰 데이터를 브라우저에 저장하는 IndexedDB라는 것도 있어요. 지금은 이름만 알아 두면 돼요.

⚠ localStorage의 한계

- 그 브라우저에만 있어요. 다른 PC나 다른 브라우저에서는 안 보여요.
- "인터넷 사용 기록 삭제"를 하거나 시크릿 창을 닫으면 사라질 수 있어요.
- 주소(출처)마다 따로예요. 바로 다음에 설명할게요.

그래서 Study-02 할 일 앱에 JSON 내보내기/가져오기를, Study-04에는 Google Drive 저장을 넣었어요. 중요한 데이터는 브라우저 밖에도 두는 게 안전해요.

1-9 출처(Origin) — "같은 사이트"의 기준

브라우저는 **프로토콜 + 도메인 + 포트**가 모두 같아야 "같은 사이트(같은 출처)"로 봐요. 쿠키와 localStorage는 출처별로 칸막이가 쳐져 있어서, 다른 출처의 데이터는 읽을 수 없어요. 이게 웹 보안의 기본 벽이에요. 은행 사이트의 로그인 쿠키를 아무 사이트나 읽으면 큰일이니까요.

🔗 그래서 Colab에서 할 일이 사라졌어요

Study-02 CLAUDE.md에 "Colab 세션이 바뀌면 저장한 할 일이 안 보인다"는 메모가 있어요. Colab이 만들어 주는 임시 주소가 **세션마다 바뀌어서 출처가 달라지기 때문**이에요. 코드가 틀린 게 아니라, 브라우저가 "다른 사이트"로 보고 다른 서랍을 연 거예요. `http://localhost:8765` 와 `http://localhost:8000` 도 포트가 다르니 서로 다른 출처예요.

CORS는 "다른 출처에 요청을 보내도 되는지"를 서버가 허락하는 규칙이에요. Study-04에서 Streamlit을 켤 때 넣은 `--server.enableCORS false` 옵션이 이것과 관련 있어요. Colab의 중계 주소가 끼어들면서 출처가 달라 보이는 문제를 피하려고 넣었어요.

1-10 캐시(Cache) — 가까운 곳에 복사본 두기

캐시는 "자주 쓰는 걸 가까운 곳에 복사해 두는 것"이에요. 멀리서 다시 가져오는 대신 가까이 둔 복사본을 쓰니 빨라요. 이 생각은 컴퓨터 곳곳에 있어요.

 **비유 — 생활관 개인 관물대**

필요할 때마다 보급창고(원래 서버)까지 가지 않고, 자주 쓰는 물건은 관물대(캐시)에 넣어 뒀요. 훨씬 빠르죠. 대신 문제가 하나 있어요. **보급창고의 물건이 새것으로 바뀌어도 관물대에는 옛날 것이 남아 있을 수 있어요.** 캐시의 장점은 속도, 단점은 "옛날 것이 보일 수 있다"는 거예요.

브라우저 캐시

브라우저는 한 번 받은 이미지, CSS, JS 파일을 저장해 두었다가 다음에 그 페이지를 열 때 재사용해요. 그래서 두 번째 방문이 더 빨라요.

 **"코드를 고쳤는데 화면이 그대로예요"**

바이브코딩을 하다 보면 정말 자주 만나는 상황이에요. 원인은 대개 브라우저가 캐시에 있던 **옛날 파일**을 보여 줘서예요. 이럴 때는:

- 강력 새로고침: `Ctrl + Shift + R` 또는 `Ctrl + F5` → 캐시를 무시하고 서버에서 새로 받아요.
- Colab이라면: 먼저 셀에서 `git pull` 로 새 코드를 받았는지 확인 → 서버를 다시 켜야 하는 앱(Streamlit 등)은 다시 켜기 → 그다음 강력 새로고침.

여러 곳의 캐시

캐시	무엇을 복사해 두나	우리와의 관계
브라우저 캐시	웹 파일(이미지, CSS, JS)	코드 고친 뒤 강력 새로고침
<code>__pycache__</code>	파이썬이 코드를 미리 번역해 둔 파일(<code>.pyc</code>)	<code>.gitignore</code> 에 넣어서 GitHub에 안 올림
npm·pip 캐시	한 번 받은 패키지	두 번째 설치가 빨라지는 이유
DNS 캐시	도메인 → IP 주소 변환 결과	1-12에서 설명
CDN	전 세계 곳곳의 서버에 웹 파일 복사본을 둬	외국 사이트도 빨리 뜨는 이유
AI 프롬프트 캐시	AI에게 매번 보내는 긴 앞부분(규칙 등)	같은 앞부분을 반복해 보낼 때 더 빠르고 싸게 처리

 **쿠키 vs 캐시 vs localStorage, 한 줄 정리**

쿠키 = 서버가 맡긴 "나 이런 사람이야" 메모(매번 서버로 따라감)

캐시 = 속도를 위해 둔 복사본(원본이 바뀌면 옛날 것일 수 있음)

localStorage = 앱이 브라우저 안에 적어 둔 데이터(서버로 안 감)

1-11 보안의 기초

HTTPS와 암호화

https의 s는 secure예요. 주고받는 내용을 암호로 바꿔서, 중간에서 누가 엿봐도 읽을 수 없게 해요. 주소창의 자물쇠 아이콘이 그 표시예요. 비밀번호나 API 키를 보내는 곳은 반드시 https여야 해요.

비밀 정보: API 키, 환경 변수, .env, Colab 보안 비밀

API 키는 서비스가 "당신이 누구인지" 확인하는 비밀 문자열이에요. 키가 새면 남이 내 돈(사용량)으로 AI를 쓸 수 있어요. 그래서 규칙이 있어요.

- **코드에 직접 쓰지 않는다.** 코드는 GitHub에 올라가니까요.
- **환경 변수**에서 읽는다. 환경 변수는 프로그램 바깥(운영체제)에 붙여 두는 이름표 달린 값이에요. 코드에는 "OPENROUTER_API_KEY라는 이름표의 값을 읽어라"만 적어요.
- 로컬 PC에서는 `.env` 파일에 적고, 이 파일은 `.gitignore` 로 GitHub에 안 올라가게 해요. 대신 "이런 이름이 필요해요"만 적은 `.env.example` 을 올려요.
- Colab에서는 **보안 비밀**(🔑)에 넣고, 셀에서 환경 변수로 옮겨요(Study-04 방식).
- 화면, `print`, 로그, 오류 메시지에 키를 출력하지 않는다. `auto-review` 점검표의 첫 줄이 바로 이거예요.
- 키는 **브라우저로 보내지 않는다.** Study-05에서 서버를 따로 둔 이유 중 하나예요. 브라우저의 JS는 누구나 F12로 볼 수 있거든요.

XSS — 입력값이 코드로 실행되는 사고

사용자가 입력한 글자를 화면에 넣을 때 `innerHTML` 로 넣으면, 입력에 HTML 코드가 섞여 있을 때 **그 코드가 실행돼요.** 예를 들어 물건 이름에 `<img src=x onerror="...">` 를 넣으면 이미지 로딩 실패를 핑계로 스크립트가 돌아요. 이것을 XSS(크로스 사이트 스크립팅)라고 해요.

쇼핑 리스트는 이름을 `textContent` 로 넣어서, 어떤 입력이든 **글자 그대로** 보여요. 오늘 테스트에서 `<img ... onerror="window.hacked=1">` 빵 을 넣고 `window.hacked` 가 생기지 않는 걸 확인한 게 바로 이 점검이에요.

개인정보

테스트 데이터는 가명(A, B, C)만 쓰고, 실명·부대 정보는 AI에게도 코드에도 넣지 않는 규칙(Study-05)도 보안의 일부예요. AI 서비스에 보낸 글은 그 회사 서버를 거친다는 걸 늘 기억하세요.

1-12 네트워크: IP, DNS, 프록시, 허용 목록

- **IP 주소:** 인터넷에 연결된 컴퓨터의 진짜 번호(예: `142.250.xx.xx`). 도메인은 사람이 외우기 쉽게 붙인 이름이에요.
- **DNS:** 도메인을 IP로 바꿔 주는 **전화번호부**. `vibeindex.ai` 를 치면 DNS에 물어서 IP를 알아낸 다음 접속해요.
- **프록시(proxy):** 내 대신 인터넷에 접속해 주는 **대리인**. 모든 요청이 프록시를 거치게 하면, 프록시가 "여기는 가도 되고 저기는 안 돼"를 정할 수 있어요.
- **허용 목록(allowlist):** 가도 되는 곳의 목록. 목록에 없으면 막아요.

🌸 오늘의 403 사건

Claude가 일하는 클라우드 컨테이너는 모든 인터넷 요청이 **프록시**를 거치고, 프록시는 **환경 설정의 허용 목록**에 있는 곳만 보내 줘요. 그래서 처음엔 vibeindex.ai에 가려다 "CONNECT tunnel failed, response 403"을 받았어요. 사용자가 환경 설정의 **Network access**에서 vibeindex.ai를 허용하자 바로 200으로 바뀌었죠. 그 와중에 www.google.com 접속 실패 메시지가 계속 보였는데, 이건 Chromium 브라우저가 켜질 때 습관처럼 구글에 연락하려다 막힌 거라 우리 작업과는 상관없었어요.

사지방 PC에서 어떤 사이트는 되고 어떤 사이트는 안 되는 것도 같은 원리예요. 부대 네트워크의 방화벽·프록시가 허용 목록으로 걸러 주는 거예요.

2부. 개발 도구의 기초

코드를 쓰는 것만큼 중요한 게 코드를 보관하고, 설치하고, 확인하는 일이에요. 여기서는 그 도구들을 볼게요.

2-1 파일, 폴더, 경로

컴퓨터의 파일은 폴더 안의 폴더 안에... 나무 모양으로 정리돼 있어요. 파일의 위치를 글자로 적은 게 **경로(path)**예요.

```
/home/user/VibeCoding/
├─ CLAUDE.md
├─ .mcp.json
├─ .claude/settings.json
├─ Study-05/
│   └─ timetable.html
└─ Study-06/
    ├─ CLAUDE.md
    ├─ .claude/
    │   └─ settings.json
    │   └─ skills/auto-review/SKILL.md
    └─ shopping-list/index.html
```

← 저장소 맨 위 (루트)

← 점(.)으로 시작 = 숨김 파일

- **절대 경로**: 맨 꼭대기(/)부터 다 적은 것. 예) /home/user/VibeCoding/Study-06/CLAUDE.md
- **상대 경로**: 지금 있는 곳에서부터 적은 것. . 은 "여기", .. 은 "한 칸 위". Study-06에서 ../Study-05/timetable.html 은 "한 칸 올라가서 Study-05 안의 파일"이에요.
- **숨김 파일**: 이름이 점(.)으로 시작하면 보통 목록에서 숨겨져요. .claude , .gitignore , .env , .mcp.json 처럼 **설정 파일**이 주로 이렇게 생겼어요. "평소엔 안 건드리는 파일"이라는 표시예요.

작업 디렉터리(작업 폴더)가 중요한 이유

프로그램은 "지금 어느 폴더에서 일하고 있는지(작업 디렉터리)"를 기준으로 상대 경로를 해석하고 설정 파일을 찾아요. Claude Code도 작업 폴더의 .claude/ 와 .mcp.json 을 읽어요. 그래서 " .mcp.json 을 Study-06 안으로 옮기면 클라우드 세션(맨 위 폴더에서 시작)이 못 찾는다"는 일이 생긴 거예요.

2-2 터미널, 셸, 명령어

터미널은 마우스 대신 **글자로 컴퓨터에 명령하는 창**이고, **셸**은 그 글자를 알아듣고 실행해 주는 프로그램이에요. 사지방에서는 쓸 수 없지만, Colab 셸에서 줄 앞에 ! 를 붙이면 셸 명령이 돼요(!git clone ...). Claude가 대화 중에 실행하는 명령(Bash)도 컨테이너의 터미널에서 도는 거예요.

명령	뜻
ls	이 폴더에 뭐가 있나 보기
cd 폴더	그 폴더로 이동

cat 파일	파일 내용 보기
python3 파일.py	파이썬 코드 실행
python3 -m http.server 8765	이 폴더를 8765번 포트로 내주는 간단한 서버 켜기
git ..., npm ..., pip ...	다음에 나올 도구들

2-3 프로그래밍 언어와 런타임

- **Python:** 읽기 쉬운 문법, 데이터·AI 분야에서 많이 써요. Study-01(숫자 인식), Study-04(AI 레시피), Study-05의 서버.
- **JavaScript:** 원래 브라우저 안에서 도는 언어. 웹 페이지의 동작을 담당해요.
- **Node.js:** JavaScript를 브라우저 밖에서도 돌릴 수 있게 해 주는 프로그램(런타임). npx , Playwright 테스트 스크립트, PPT를 만든 make_slide.js 가 Node.js로 돌아왔어요.

런타임은 "코드를 실제로 돌려 주는 프로그램"이에요. 파이썬 코드는 파이썬 런타임이, JS는 브라우저나 Node.js가 돌려요. 오늘 "Node.js가 안 깔려서 그런가요?"라고 물어보셨을 때 node --version 으로 v22가 있는 걸 확인했죠. 런타임이 있는지부터 보는 게 좋은 첫 점검이에요.

2-4 패키지, 라이브러리, 패키지 관리자

라이브러리(패키지)는 남이 만들어 둔 코드 묶음이에요. 모든 걸 직접 만들 필요 없이 가져다 써요. 예) 웹 요청은 requests , 화면은 Streamlit·Gradio , 브라우저 조종은 Playwright , PPT는 pptxgenjs .

패키지 관리자는 패키지를 찾아서 설치해 주는 프로그램이에요.

언어	관리자	참고	설치 명령	설치되는 곳
Python	pip	PyPI	pip install requests	파이썬 폴더
JavaScript	npm	npm 레지스트리	npm install pptxgenjs	그 폴더의 node_modules/

- **npx:** "설치해 두지 않고 한 번만 받아서 바로 실행"하는 명령이에요. npx vibeindex add ... , npx @playwright/mcp 가 이 방식이었어요.
- **의존성(dependency):** 내 코드가 돌아가려면 필요한 다른 패키지. 패키지가 또 다른 패키지를 필요로 해서 줄줄이 설치되기도 해요.
- **버전 고정:** @playwright/mcp@0.0.82 처럼 버전을 박아 두면, 나중에 새 버전이 나와도 오늘 확인한 그대로 동작해요. @latest (항상 최신)는 편하지만 어느 날 갑자기 동작이 바뀔 수 있어요.
- node_modules/ 는 크고 다시 설치하면 생기는 폴더라 GitHub에 올리지 않아요. PPT 스크립트도 그래서 패키지는 저장소 밖에 설치했어요.

🔥 버전이 안 맞으면 생기는 일

Playwright MCP를 처음 켜올 때 "chromium-1246이 없다"는 오류가 났어요. 새 Playwright는 브라우저 1246판을 기대했는데, 컨테이너에는 1194판이 깔려 있었거든요. 그래서 `--executable-path` 로 "이미 있는 이 브라우저를 써"라고 알려 줬어요. "**프로그램이 기대하는 버전 ≠ 실제로 있는 버전**"은 개발에서 가장 흔한 문제 중 하나예요.

라이브러리 vs 프레임워크: 라이브러리는 내가 필요할 때 불러 쓰는 도구(공구함)이고, 프레임워크는 틀이 정해져 있어서 내가 그 틀에 코드를 끼워 넣는 것(조립식 가구 설계도)이에요. Streamlit, Gradio는 프레임워크에 가까워요.

2-5 Git과 GitHub

Git은 코드의 **변경 이력**을 저장하는 **프로그램**이고, **GitHub**는 Git 저장소를 인터넷에 보관하고 공유하는 **웹사이트**예요.

📖 비유 — 게임 세이브 포인트

커밋은 세이브 포인트예요. "여기까지 잘 됐다" 싶을 때 저장하면, 나중에 망가져도 그 지점으로 돌아갈 수 있어요. 각 세이브에는 메모(커밋 메시지)를 붙여요. **브랜치**는 평행 세계예요. 본편(main)을 건드리지 않고 다른 세계에서 실험하다가, 괜찮으면 본편에 합쳐요(**머지**).

용어	뜻	우리 경우
저장소(repository, repo)	Git이 관리하는 프로젝트 폴더 하나	<code>sejoor12345-commits/VibeCoding</code>
커밋(commit)	변경을 메시지와 함께 저장	<code>Study-06: 쇼핑 리스트 앱 추가 ...</code> (규칙: <code>Study-NN: 한국어 요약</code>)
브랜치(branch)	독립된 작업 줄기	Claude는 <code>claude/wizardly-allen-jp6a7m</code> 브랜치에서 작업
원격(remote)	인터넷에 있는 저장소 사본	GitHub의 저장소 (<code>origin</code> 이라고 불러요)
push	내 커밋을 원격으로 올리기	Claude가 작업 끝에 올림
clone	원격 저장소를 통째로 처음 받아오기	Colab에서 <code>!git clone ...</code>
pull	원격의 새 커밋을 받아와 합치기	Colab에서 <code>git pull</code> 후 새로고침
PR(Pull Request)	"내 브랜치를 본편에 합쳐 주세요" 요청서	필요할 때 Claude에게 만들어 달라고 할 수 있음
<code>.gitignore</code>	Git이 무시할 파일 목록	<code>.env</code> , <code>__pycache__</code> / , <code>.playwright-mcp/</code>

📌 기억할 것

클라우드 컨테이너는 **임시**예요. 세션이 끝나면 컨테이너가 치워지고, 커밋·push하지 않은 변경은 함께 사라져요. 그래서 Claude가 매번 "커밋하고 올렸어요"라고 보고하는 거예요.

2-6 로컬, 클라우드, 컨테이너

- **로컬**: 내 앞에 있는 컴퓨터. 책은 로컬 PC 작업을 전제로 설명해요.
- **클라우드**: 인터넷 너머 회사의 컴퓨터를 빌려 쓰는 것. Colab, Claude Code 웹 세션, Hugging Face Spaces(Study-01 웹 버전 배포처)가 모두 클라우드예요.
- **컨테이너**: 클라우드 컴퓨터 안에 만든 **독립된 임시 작업실**이에요. 필요한 도구가 깔린 상태로 금방 만들어지고, 다 쓰면 통째로 치워져요. 그래서 오늘 LibreOffice Impress나 한글 글꼴을 설치해도 다음 세션에는 없어요.
- **배포(deploy)**: 만든 앱을 다른 사람도 접속할 수 있는 곳에 올려 두는 것. Study-01 웹 버전을 Hugging Face Spaces에 올린 게 배포예요.

🔥 환경 설정(Environment)

클라우드 세션에는 "환경"이라는 설정 묶음이 있어요. **네트워크 허용 목록**(오늘 vibeindex.ai를 추가한 곳), 환경 변수, 시작할 때 실행할 스크립트 등을 정해요. 세션마다 새 컨테이너가 이 설정대로 만들어져요.

2-7 테스트와 코드 품질

테스트는 "이 코드가 기대대로 동작하는지" 확인하는 일이고, **자동 테스트**는 그 확인을 코드로 적어 두어 언제든지 다시 돌릴 수 있게 한 거예요.

- **단언(assertion)**: "이 값은 반드시 이래야 한다"는 문장. 예) "우유를 추가하면 목록은 ['우유']여야 한다." 맞으면 , 틀리면 .
- **경계값·엣지 케이스**: 빈 목록, 0, 빈칸 입력, 아주 긴 글자, 깨진 데이터처럼 **평소에 잘 안 일어나지만 버그가 숨어 있기 쉬운 상황**. 오늘 28개 테스트의 절반 가까이가 이런 것들이었어요.
- **회귀(regression)**: 새 기능을 넣다가 예전에 되던 게 망가지는 것. 자동 테스트가 있으면 바로 잡아내요. Study-05의 `runTests()` 221개가 이런 안전망이에요.
- **린터(linter)**: 코드를 실행하지 않고 읽어서 실수와 스타일 문제를 찾아 주는 도구. 우리 저장소에는 없어서 `CLAUDE.md` 규칙과 `auto-review` 스킬이 그 역할을 해요.
- **문법 검사**: `python3 -m py_compile`, `node --check` 처럼 "문법이 맞는지만" 빠르게 보는 것.
- **코드 리뷰**: 사람(또는 AI)이 코드를 읽고 문제를 찾는 것. `auto-review` 스킬, Study-04의 `code-reviewer` 에이전트.

Playwright와 헤드리스 브라우저

Playwright는 코드로 브라우저를 조종하는 도구예요. "이 주소를 열어라 → 입력칸에 '우유'를 써라 → 추가 버튼을 눌러라 → 목록에 우유가 있는지 확인해라"를 사람 대신 해 줘요. **헤드리스(headless)**는 화면 없이 브라우저를 돌리는 방식이에요. 컨테이너에는 모니터가 없으니 화면 없이 돌리고, 필요하면 스크린샷만 찍어요.

2-8 오류 메시지 읽는 법

오류 메시지는 무서워 보이지만 대부분 **무엇이 · 어디서 · 왜**를 말해 주고 있어요. 오늘 실제로 만난 오류로 연습해 볼게요.

오류 메시지	실제 뜻	해결
No files found in the skill directory.	스킬 폴더는 찾았는데 파일 목록을 못 받음 (GitHub API가 403으로 막힘)	git clone으로 직접 받아서 복사
CONNECT tunnel failed, response 403	프록시가 이 사이트로 가는 걸 막음	환경 설정에서 허용 목록에 추가
Chromium distribution 'chrome' is not found	"크롬"을 찾는데 없음 (Chromium만 있음)	--browser chromium
Browser ... is not installed; expected ... chromium-1246	기대한 버전의 브라우저가 없음	--executable-path 로 있는 브라우저 지정
Access to "file:" protocol is blocked	보안상 file:// 주소 금지	로컬 서버를 띄워 http://localhost로 열기
source file could not be loaded	PPT를 읽을 부품(Impress)이 LibreOffice에 없었음	libreoffice-impress 설치

문제를 알릴 때의 공식

"무엇을 했더니 → 어떻게 됐고 → 원래는 이렇게 되길 기대했다" + 오류 메시지 원문 복사. 이 세 가지만 있으면 사람이든 AI든 훨씬 빨리 도와줄 수 있어요. browser-check 스킬의 보고 형식도 이 공식을 따라요.

3부. AI의 기초

Study-01의 숫자 인식부터 Study-04·05의 OpenRouter, 그리고 Claude Code 자신까지. AI가 무엇이고 어떻게 불러 쓰는지 볼게요.

3-1 AI, 머신러닝, 딥러닝, LLM

AI — 사람처럼 판단하는 프로그램 전반

머신러닝 — 데이터로 규칙을 스스로 배움

딥러닝 — 여러 층의 인공 신경망

LLM — 글을 아주 많이 배운 거대 모델
Claude, GPT, Gemini, space-bunny-alpha ...

- **머신러닝**: 사람이 규칙을 일일이 짜는 대신, **예시 데이터를 많이 보여 주고 규칙을 스스로 찾게** 하는 방식.
- **학습(training)과 추론(inference)**: 데이터를 보며 규칙을 익히는 단계가 학습, 익힌 규칙으로 새 문제에 답하는 단계가 추론이에요. (3-3의 "추론 모델"의 추론과는 뜻이 조금 달라요.)
- **모델**: 학습이 끝난 결과물. 규칙이 숫자 문치로 저장된 파일이에요.

🌟 Study-01 = 머신러닝 실습

손글씨 숫자 70,000장(MNIST 데이터셋)으로 MLPClassifier (작은 신경망)를 학습시키고, Gradio 캔버스에 그린 숫자를 추론했어요. `load_or_train_model()` 은 "저장된 모델이 있으면 불러오고, 없으면 학습해라"라는 뜻이에요. 학습은 오래 걸리니 한 번 한 결과를 저장해 재사용하는 거죠. 이것도 일종의 캐시예요!

3-2 LLM이 동작하는 방식: 토큰, 컨텍스트, 프롬프트

LLM(대규모 언어 모델)은 아주 단순하게 말하면 **"지금까지의 글 다음에 올 말을 예측하는 기계"**예요. 엄청나게 많은 글을 학습해서, 질문 다음에 올 법한 좋은 답을 한 조각씩 이어 붙여요.

- **토큰(token)**: LLM이 글을 읽고 쓰는 단위. 단어 하나, 단어의 일부, 한글 한두 글자쯤이에요. AI 요금과 길이 제한은 대부분 토큰 수로 계산해요.
- **컨텍스트 윈도우**: LLM이 **한 번에 볼 수 있는 글의 양**(토큰 수). 대화 내용, 읽은 파일, 규칙이 모두 이 안에 들어가야 해요.
- **프롬프트**: AI에게 주는 지시나 질문. **시스템 프롬프트**는 대화 전체에 깔리는 기본 규칙("너는 요리사야, JSON으로만 답해")이에요.
- **환각(hallucination)**: 그럴듯하지만 틀린 내용을 자신 있게 말하는 것. 그래서 AI 결과는 **검증**이 필요해요. Study-05에서 AI에게 근무표를 통째로 맡기지 않은 이유이기도 해요.

비유 — 책상 크기

컨텍스트 윈도우는 **책상 크기**예요. 책상이 아무리 커도 무한하진 않아서, 온갖 자료를 다 펼쳐 두면 정작 중요한 게 묻혀요. 4부의 스킬과 서브에이전트는 이 책상을 아껴 쓰는 방법이기도 해요. 스킬은 "필요할 때만 책을 꺼내 펼치기", 서브에이전트는 "옆 책상의 동료에게 맡기고 결과 요약만 받기"예요.

3-3 추론 모델과 멀티모달

- **추론(reasoning) 모델**: 답하기 전에 속으로 단계별로 생각하는 모델. Study-04의 사진 인식 모델은 생각한 과정을 `<think>...</think>` 로 감싸서 내보내서, 우리 코드가 그 부분을 지우고 답만 쓰게 했어요.
- **멀티모달**: 글뿐 아니라 이미지·음성 등을 함께 다루는 것. Study-04에서 냉장고 사진을 보내 재료 목록을 받은 게 멀티모달 활용이에요. 이미지는 base64라는 방식으로 글자로 바꿔 JSON에 담아 보냈어요.

3-4 API로 AI 쓰기 (Study-04 · 05)

API는 프로그램끼리 대화하는 **창구와 약속**이에요. "이 주소로, 이런 JSON을 보내면, 이런 JSON을 돌려줄게"라는 규칙이죠. 그 주소 하나하나를 **엔드포인트**라고 불러요.

POST `https://openrouter.ai/api/v1/chat/completions`

헤더: `Authorization: Bearer (API 키)`

본문: `{`

```
  "model": "stealth/space-bunny-alpha",
  "messages": [
    { "role": "system", "content": "너는 요리사야. JSON으로만 답해." },
    { "role": "user", "content": "계란, 대파, 밥으로 뭘 만들 수 있어?" }
  ]
}
```

응답: `{ "choices": [ { "message": { "content": "{\\"title\\": \\"대파 계란볶음밥\\", ...}" } } ] }`

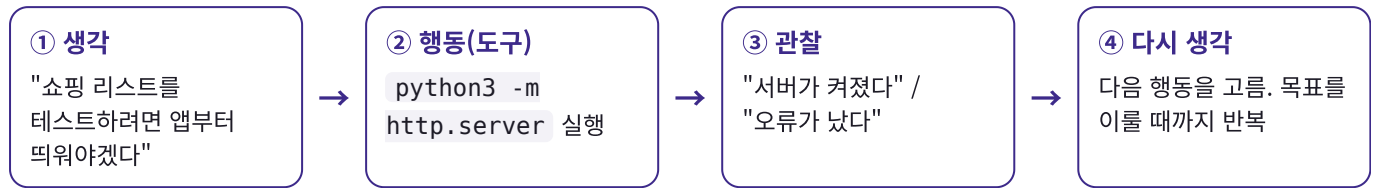
- **OpenRouter**: 여러 회사의 AI 모델을 한 창구에서 골라 쓸 수 있게 해 주는 중개소예요. 코드에서 모델 이름만 바꾸면 다른 AI를 쓸 수 있어요. 그래서 모델 이름을 상수 하나(`openrouter_client.py`)에 모아 두는 규칙을 만들었어요.
- **타임아웃**: 너무 오래 답이 없으면 포기하는 시간 제한. 무한정 기다리면 앱이 멈춘 것처럼 보이니까요.
- **재시도·친절한 오류 메시지**: 401/429/5xx마다 사용자에게 무엇을 하면 되는지 알려 주는 것. Study-04에서 배운 교훈을 Study-05 에이전트 설명서에도 옮겨 적었어요.

하이브리드 설계 — Study-05에서 가장 중요했던 결정

AI에게 근무표 전체를 쓰게 하면 "야간 다음 날 주간" 같은 규칙을 자주 어겨요. 그래서 **AI는 잘하는 일(사람의 자유로운 말을 정해진 규칙 형식으로 바꾸기)만 하고, 계산과 규칙 지키기는 기존 코드(규칙 엔진)가 하게** 나눴어요. "AI에게 무엇을 맡기고 무엇을 맡기지 않을지"를 정하는 게 AI 앱 설계의 핵심이에요.

3-5 에이전트 — 도구를 쓰는 AI

보통의 챗봇은 **말만** 해요. **에이전트**는 LLM에게 **도구**(파일 읽기, 명령 실행, 웹 검색, 브라우저 조종...)를 쥐여 주고, 목표를 이룰 때까지 **생각 → 행동 → 결과 보기 → 다시 생각**을 반복하게 한 거예요.



Claude Code가 바로 에이전트예요. 오늘 Playwright를 연결할 때 "chrome이 없네 → chromium으로 → 버전이 안 맞네 → 경로 지정 → file://이 막혔네 → 서버로"처럼 스스로 오류를 보고 다음 수를 고른 과정이 에이전트의 반복 그 자체예요. 4부는 이 에이전트에게 **규칙, 설명서, 동료, 도구**를 더해 주는 방법들이예요.

4부. Claude Code 확장 체계

이 노트의 핵심이에요. CLAUDE.md, 명령어, 스킬, 서브에이전트, MCP, 혹은 플러그인, 마켓플레이스가 각각 무엇이고 어떻게 이어지는지 볼게요.

4-1 Claude Code를 쓰는 여러 장소

장소	어디서 돌아가나	특징
터미널(CLI)	내 PC	가장 기본. <code>/plugin</code> 같은 모든 명령을 쓸 수 있음
데스크톱 앱 · VS Code 등 IDE	내 PC	화면이 더 편한 CLI라고 보면 됨
웹 (claude.ai/code)	클라우드 컨테이너	우리가 쓰는 곳. 내 PC에 아무것도 없어도 됨. 대신 일부 기능(<code>/plugin</code> 등)이 없음

세션은 대화 한 판이에요. 세션이 시작될 때 Claude Code는 작업 폴더의 설정들(CLAUDE.md, .mcp.json, settings.json, 스킬 목록)을 읽어요. 그래서 "새 세션부터 적용돼요"라는 말이 자주 나왔어요. 실제로 Playwright 연결은 설정을 만든 뒤의 세션에서야 도구 목록에 나타났어요.

4-2 CLAUDE.md — 프로젝트 규칙 수첩

무엇인가: Claude Code가 세션을 시작할 때 **자동으로 읽는 메모 파일**이에요. 프로젝트 설명, 규칙, 지금까지의 결정과 교훈을 적어 뒹요. Claude는 대화가 끝나면 기억을 잃지만, CLAUDE.md는 저장소에 남아서 다음 세션의 Claude에게 전해져요.

비유 — 인수인계서

근무 교대할 때 "특이사항: ○○는 이렇게 처리할 것"을 적어 두는 인수인계서예요. 다음 근무자(다음 세션의 Claude)는 이걸 먼저 읽고 일을 시작해요.

- **계층:** 저장소 맨 위 `CLAUDE.md` (공통 규칙)와 폴더별 `Study-NN/CLAUDE.md` (그 폴더 규칙)가 함께 쓰여요.
- **우리 규칙의 예:** "사용자는 사지방에서 Colab을 쓴다 → 터미널 명령 대신 Colab 셀로 안내", "커밋 메시지는 `Study-NN: 한국어 요약`", "API 키는 환경 변수로만", "에이전트를 만들 때는 도구 권한과 모델을 먼저 물어본다".
- **언제 쓰나:** 항상 지켜야 하는 것, 모든 작업에 필요한 배경지식.

4-3 커스텀 명령어 (Study-03)

무엇인가: 자주 쓰는 지시를 파일로 저장해 두고 `/이름` 으로 부르는 **단축 명령어**예요.

- **위치:** `.claude/commands/이름.md` (예: `Study-03/.claude/commands/quiz-add.md` → `/quiz-add`)
- **누가 부르나:** 사람이 직접 `/quiz-add` 라고 입력할 때만 실행돼요.

- **인자:** `/quiz-add` 역사 처럼 뒤에 붙인 말을 파일 안의 `$ARGUMENTS` 자리에 넣을 수 있어요.
- **언제 쓰나:** "내가 원하는 순간에, 정해진 절차를 똑같이" 실행하고 싶을 때.

비유 — 단축 다이얼

1번을 길게 누르면 늘 같은 곳으로 전화가 걸려요. 내가 누르기 전에는 절대 안 걸려요.

4-4 스킬 (Study-06)

무엇인가: 특정 일을 할 때 따르는 **설명서 묶음**이에요. 명령어와 가장 큰 차이는 **Claude가 상황을 보고 알아서 꺼내 쓴다**는 점이에요.

```
.claude/skills/auto-review/
├── SKILL.md           ← 필수. 이름·설명·절차
├── checklist.md       ← 부속 자료 (필요할 때만 읽음)
└── report-template.md ← 부속 자료

SKILL.md 맨 위:
---
name: auto-review
description: 코드를 자동으로 리뷰한다 ... "리뷰해줘", "점검해줘"라고 하거나,
            코드를 새로 쓰거나 고친 작업을 마쳤을 때 사용한다.
allowed-tools: Read, Grep, Glob, Bash(git diff:*), ... mcp__playwright__*
---
(아래는 본문: 1단계 대상 정하기 → 2단계 규칙 읽기 → 3단계 점검 → 4단계 보고)
```

- **description이 방아쇠예요.** Claude는 평소에 스킬의 **이름과 설명만** 알고 있다가, 대화가 설명과 맞으면 그때 본문을 펼쳐 읽어요. 오늘 쇼핑 리스트를 다 만들자 "코드를 새로 쓴 작업을 마쳤을 때"라는 설명에 맞아서 `auto-review` 가 자동으로 실행된 게 그 장면이에요.
- **점진적 공개(progressive disclosure):** 평소엔 제목만 → 필요하면 본문 → 본문이 가리키면 부속 파일. 책상 (컨텍스트)을 아끼는 설계예요. 그래서 점검표를 `checklist.md` 로 따로 뺐어요.
- **allowed-tools:** 이 스킬을 쓰는 동안 **묻지 않고 써도 되는 도구** 목록이에요. "리뷰는 읽기 전용"이라는 성격에 맞게 파일 수정 도구는 넣지 않았어요.
- **/이름으로도 부를 수 있어요.** `/vibeindex` 처럼요. 그래서 요즘은 명령어와 스킬의 경계가 흐려지는 중이에요. 차이는 "Claude가 스스로 꺼내 쓸 수 있느냐"예요.
- **스크립트도 담을 수 있어요.** `webapp-testing`, `pptx` 스킬처럼 실행할 코드와 예제까지 한 폴더에 넣을 수 있어요.

비유 — 업무 매뉴얼 책장

직원(Claude)은 책장에 꽂힌 매뉴얼의 **제목**은 다 알아요. "코드 리뷰" 일이 들어오면 그 매뉴얼을 꺼내 펼치고, 부록(점검표)이 필요하면 부록까지 봐요. 누가 "매뉴얼 봐"라고 시키지 않아도요.

🔗 우리 스킬 세 개

- `auto-review` : 코드 리뷰. 읽기 전용. 웹 화면이면 `browser-check` 를 따라 브라우저로도 확인.
- `browser-check` : Playwright로 앱을 열어 직접 눌러 보는 절차.
- `vibeindex` : 외부에서 받은 스킬. 프로젝트를 분석하고 `vibeindex.ai`에서 도구를 추천.

스킬이 **다른 스킬을 가리킬 수도** 있어요(`auto-review` → `browser-check`). 작은 설명서를 조합해서 큰 일을 하게 만드는 거예요.

4-5 서브에이전트 (Study-04 · 05)

무엇인가: 특정 역할을 맡은 **또 한 명의 Claude**예요. 메인 Claude가 일을 맡기면, 서브에이전트는 **자기만의 책상(별도 컨텍스트)**에서 일하고 **결과 보고서만** 돌려줘요.

```
.claude/agents/code-reviewer.md
---
name: code-reviewer
description: 전문 코드 품질 검토자 ... "코드 리뷰"를 요청할 때 사용한다.
tools: Read, Grep, Glob, Bash          ← 쓸 수 있는 도구 (없으면 전부)
model: inherit                          ← 부른 쪽과 같은 모델
---
(아래는 그 에이전트의 성격과 일하는 방식, 보고 형식)
```

- **별도 컨텍스트:** 서브에이전트가 파일 50개를 읽어도 메인 대화의 책상은 어지럽혀지지 않아요. 요약 보고만 받으니까요.
- **도구 제한:** `code-reviewer` 에는 Edit/Write를 주지 않아서 **구조적으로** 파일을 못 고쳐요. 스킬의 "고치지 마세요"라는 글보다 더 확실한 제한이에요.
- **사용자에게 직접 못 물어요.** 그래서 Study-05 에이전트들은 보고서에 "사용자에게 물어볼 것" 칸을 두었어요.
- **팀 구성:** Study-05에서는 `product-manager` → `ai-integration-specialist` / `backend` / `frontend` → `qa-engineer` 순서로 역할을 나눴어요. 실제 회사의 개발 팀과 같은 구조예요.

📌 스킬 vs 서브에이전트

스킬은 "**나(Claude)에게 주는 매뉴얼**"이고, 서브에이전트는 "**일을 맡길 다른 직원**"이에요. 같은 코드 리뷰라도, 대화 흐름 안에서 바로 하고 싶으면 스킬(`auto-review`), 따로 떼어 맡기고 결과만 받고 싶으면 서브에이전트(`code-reviewer`)가 어울려요.

4-6 MCP — AI에게 새 도구를 쏘는 규격

MCP(Model Context Protocol)는 AI 앱과 바깥 도구를 연결하는 **공통 규격**이에요. Anthropic이 만들어 공개했고, 지금은 여러 회사의 AI 앱이 함께 써요.

🔌 비유 — USB-C

예전엔 기기마다 충전기 모양이 달랐죠. USB-C처럼 규격이 하나로 정해지면, 어떤 기기든 같은 선으로 꽂을 수 있어요. MCP도 마찬가지로, 도구 쪽(MCP 서버)이 규격대로 만들어져 있으면 Claude Code 같은 AI 앱(MCP 클라이언트)에 바로 꽂혀요.

- **MCP 서버**: 도구를 제공하는 프로그램. 예) `@playwright/mcp` 는 `browser_navigate` , `browser_click` , `browser_snapshot` 등 **도구 25개**를 제공했어요.
- **연결 방법**: 작업 폴더의 `.mcp.json` 에 "이 명령으로 서버를 켜라"를 적어요.
- **도구 이름**: Claude 쪽에서는 `mcp__서버이름__도구이름` 으로 보여요. 예) `mcp__playwright__browser_click` . 스킬의 `allowed-tools` 에 `mcp__playwright__*` 라고 적은 게 "playwright 서버의 모든 도구"라는 뜻이에요.
- **커넥터**: `claude.ai`에서 연결하는 Notion, GitHub, Google Drive 같은 것도 MCP로 만들어져 있어요.

```
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": [
        "-y",
        "@playwright/mcp@0.0.82",
        "--headless",
        "--isolated",
        "--browser",
        "chromium",
        "--executable-path",
        "/opt/pw-browsers/chromium",
        "--output-dir",
        ".playwright-mcp"
      ]
    }
  }
}
```

← 이 프로그램을
← 이 패키지(버전 고정)로
← 화면 없이
← 매번 깨끗한 브라우저로
← 이미 깔린 브라우저
← 스크린샷 저장 위치

🔗 스킬과 MCP의 관계

MCP는 **손(도구)**을 주고, 스킬은 그 손을 **어떻게 쓸지(절차)**를 알려 줘요. Playwright MCP만 있으면 Claude는 브라우저를 조종할 수 있지만, "무엇을 어떤 순서로 확인하고 어떻게 보고할지"는 `browser-check` 스킬이 정해 줘요. 둘이 함께일 때 가장 쓸모 있어요.

4-7 훅(Hook) — 정해진 순간에 반드시 실행되는 규칙

무엇인가: "이런 일이 생기면 이 명령을 실행하라"를 설정 파일에 적어 두는 것이에요. 우리가 아직 직접 만들어 보지 않은 유일한 부품이에요.

- **언제**: 도구를 쓰기 직전, 도구를 쓴 직후, 세션이 시작될 때, Claude가 답을 마쳤을 때, 사용자가 메시지를 보냈을 때 등(이런 순간을 "이벤트"라고 해요).
- **예시**: 파일을 고친 직후 자동으로 문법 검사 돌리기 / `.env` 파일을 건드리려 하면 막기 / 세션이 시작될 때 필요한 패키지 설치하기 / 일이 끝나면 알림 보내기.
- **위치**: `settings.json` 의 `hooks` 항목.

🔴 **혹이 특별한 이유 — "반드시"**

스킬과 CLAUDE.md는 Claude가 읽고 따르는 글이라, 상황에 따라 판단이 들어가요. 혹은 Claude가 아니라 **Claude Code 프로그램이 기계적으로 실행**해요. 그래서 "무조건 매번" 해야 하는 일은 **혹**이 맞아요. "앞으로 파일을 저장할 때마다 검사해 줘"처럼 "**~할 때마다**"라는 부탁은 **혹**으로 만드는 거예요.

📖 **비유 — 자동문 센서**

경비병에게 "사람이 오면 문 열어 줘"라고 부탁하는 건 스킬이에요(대개 잘하지만 사람이 하는 일). 자동문 센서는 **혹**이에요. 누가 오면 판단 없이 무조건 열려요.

4-8 settings.json과 권한, 그리고 범위

settings.json 은 Claude Code의 **설정 파일**이에요. 권한(무엇을 묻지 않고 해도 되는지), **혹**, **켜** 둘 MCP 서버와 플러그인 등을 적어요. **어디에 두느냐(범위, scope)**에 따라 적용 대상이 달라져요.

범위	파일 위치	누구에게 적용	GitHub에 올라가나
user	~/ .claude/settings.json (내 PC)	나의 모든 프로젝트	아니요
project	.claude/settings.json (저장소 안)	이 저장소를 쓰는 모든 사람	예
local	.claude/settings.local.json	이 저장소에서 나만	아니요
managed	회사가 관리	조직 전체 (가장 강함)	—

🌸 **우리가 만든 settings.json 두 개**

- 저장소 맨 위 .claude/settings.json : "enabledMcpJsonServers": ["playwright"] — ".mcp.json의 playwright 서버를 켜라"
- Study-06/.claude/settings.json : extraKnownMarketplaces (마켓플레이스 등록) + enabledPlugins (document-skills 켜기). "install for all collaborators on this repository" = **project 범위**로 설치한 것과 같은 내용이에요.

권한 모드도 알아 두면 좋아요. Claude는 위험할 수 있는 행동(파일 수정, 명령 실행, 외부 전송) 전에 허락을 구해요. 오늘 외부 스킬 파일을 저장소에 넣으려다 안전 검사에 막힌 것도 이 장치예요. 인터넷에서 받은 "설명서"는 Claude가 그대로 따르게 되니, **사람이 한 번 확인하도록** 막아 둔 거예요. 사용자님이 "넣어도 돼"라고 허락한 뒤에 진행했죠.

4-9 플러그인과 마켓플레이스

플러그인은 지금까지 본 부품(스킬, 명령어, 서브에이전트, MCP 서버, **혹**)을 **한 꾸러미로 묶어 설치하기 쉽게** 만든 것이에요. **마켓플레이스**는 플러그인 **목록(가게)**이에요.

📖 비유 — 도시락과 도시락 가게

반찬(부품)을 하나하나 만들 수도 있지만, 잘 만든 반찬을 담은 도시락(플러그인)을 사 오면 편해요. 도시락 가게(마켓플레이스)를 여러 곳 알수록 고를 수 있는 도시락이 많아져요. 하지만 **어느 가게 도시락이든 담긴 반찬의 종류는 같아요**: 스킬, 명령어, 에이전트, MCP, 혹.

마켓플레이스 (목록, 여러 개 추가 가능)

```
├─ claude-plugins-official    ← PC 터미널에서 처음 켤 때 자동으로 추가됨
│   ├── playwright          = MCP 서버 1개
│   ├── commit-commands     = 명령어 여러 개
│   └── ...
└─ anthropic-agent-skills    ← /plugin marketplace add anthropics/skills 로 추가
    ├── document-skills      = 스킬 4개 (pptx, docx, xlsx, pdf)
    ├── example-skills       = 스킬 12개 (frontend-design, webapp-testing ...)
    └── ...
```

- **마켓플레이스의 정체**: 대개 GitHub 저장소 하나예요. 안의 `.claude-plugin/marketplace.json` 에 "이 가게에는 이런 플러그인이 있어요" 목록이 적혀 있어요.
- **플러그인의 정체**: 폴더 하나. `.claude-plugin/plugin.json` (이름표) + 부품들(`skills/` , `commands/` , `agents/` , `.mcp.json` , `hooks`).
- **설치 명령(PC 터미널)**: `/plugin marketplace add 저장소` → `/plugin install 플러그인@마켓플레이스` → 범위 선택(user / project / local).

🔥 클라우드 세션에서는 어떻게 했나

클라우드 세션에는 `/plugin` 이 없고, 설정 파일에 플러그인을 적어도 설치하지 않아요. 그래서 우리는:

1. **플러그인을 열어 부품을 꺼냈어요**. playwright 플러그인 = MCP 서버 하나 → `.mcp.json` 에 직접 등록.
2. **설치 기록은 남겼어요**. document-skills는 settings.json에 적어 두어, 나중에 PC에서 열면 설치를 제안받게 했어요.
3. **같은 기능은 이미 있는 걸 썼어요**. 이 세션에는 pptx·pdf 스킬이 계정에 들어 있어서, PPT와 이 PDF 노트를 그걸로 만들었어요.

플러그인이 "부품 꾸러미"라는 걸 알면, 설치 도구가 없어도 부품만 꺼내 쓸 수 있어요.

4-10 한눈 비교표

부품	한 줄 정의	누가 · 언제 커나	파일 위치	우리 예
CLAUDE.md	프로젝트 규칙 수첩	세션 시작 때 항상 자동으로 읽힘	CLAUDE.md	맨 위 + Study별
커스텀 명령어	단축 명령	사람이 /이름 입력할 때	<code>.claude/commands/</code>	<code>/quiz-add</code>
스킬	상황별 업무 매뉴얼	Claude가 설명과 맞을 때 스스로 (또는 사람이 /이름)	<code>.claude/skills/이름/SKILL.md</code>	auto-review, browser-check

서브에이전트	일을 맡길 다른 직원	Claude가 맡기거나 사람이 지목할 때	<code>.claude/agents/</code>	code-reviewer, qa-engineer
MCP 서버	새 도구(손)를 꽂는 규격	세션 시작 때 연결, 도구는 Claude가 필요할 때 사용	<code>.mcp.json</code>	playwright
훅	정해진 순간에 무조건 실행	프로그램이 이벤트마다 자동으로	<code>settings.json</code> 의 hooks	(아직 없음)
settings.json	권한·연결·플러그인 설정	세션 시작 때 읽힘	<code>.claude/settings.json</code>	2개
플러그인	위 부품들의 꾸러미	설치하면 부품들이 각자 방식으로 동작	(설치됨)	document-skills
마켓플레이스	플러그인 목록 (가게)	추가하면 설치 가능한 목록이 늘어남	GitHub 저장소	anthropics/skills

4-11 언제 무엇을 쓰나 — 결정 가이드

이런 걸 원하면	이걸 쓰세요
모든 작업에서 늘 지켜야 할 규칙, 배경 정보	CLAUDE.md
내가 원하는 순간에 정해진 절차를 똑같이	커스텀 명령어
특정 종류의 일이 생기면 Claude가 알아서 그 방식으로	스킬
큰 일을 따로 떼어 맡기고 결과만 받기 / 도구를 확실히 제한	서브에이전트
Claude가 원래 못 하는 일(브라우저 조종, Notion 읽기 등)을 하게	MCP 서버
"~할 때마다 무조건" 실행	훅
위의 것들을 남과 쉽게 나누거나 남이 만든 걸 한 번에 받기	플러그인 + 마켓플레이스

4-12 클라우드 세션 vs PC 터미널

항목	PC 터미널(CLI)	클라우드 세션(우리)
CLAUDE.md, 스킬, 명령어, 에이전트	✅	✅ (저장소에 들어 있으면)
저장소의 <code>.mcp.json</code>	✅	✅ (세션이 시작하는 맨 위 폴더에 있어야 함)
<code>/plugin</code> 명령, 플러그인 설치	✅	❌ → 부품을 꺼내 직접 넣기
내 PC에 설치한 것들	✅	❌ (다른 컴퓨터니까)
인터넷 접속	자유	환경의 허용 목록만
설치한 프로그램	계속 남음	세션이 끝나면 사라짐 (컨테이너)

5부. 실습 되짚기

Study-01부터 06까지 무엇을 만들었고, 거기서 어떤 개념을 만났는지 다시 봐요. 개념이 "실제로 어디에 쓰였는지"와 연결되면 오래 기억돼요.

01 Study-01 · 손글씨 숫자 인식기

- **만든 것:** 캔버스에 숫자를 그리면 맞는 앱. PC용(`desktop_version`)과 인터넷 공개용(`web_version` , Hugging Face Spaces) 두 벌.
- **만난 개념:** 머신러닝, 학습과 추론, 데이터셋(MNIST), 모델 저장·재사용, Gradio(화면 프레임워크), 배포, GitHub Actions(자동으로 `.exe` 만들기).
- **교훈:** 같은 핵심 로직도 실행 환경(PC / 웹)에 따라 포장이 달라진다.

02 Study-02 · 할 일 관리 앱

- **만든 것:** 추가·수정·삭제, 카테고리, 필터, 검색, 정렬, 드래그 순서 바꾸기, 되돌리기, 내보내기/가져오기, 다크 모드까지 갖춘 앱.
- **만난 개념:** HTML/CSS/JS 역할 분담, DOM, 이벤트, localStorage, JSON, 반응형 디자인(`@media`), 출처(Colab 주소가 바뀌면 데이터가 안 보임).
- **교훈:** 브라우저 저장은 편하지만 그 브라우저·그 주소에만 있다 → 내보내기 기능이 필요하다.

03 Study-03 · 상식 퀴즈 게임 + 커스텀 명령어

- **만든 것:** 카테고리별·전체 도전·스피드 모드 퀴즈, 콤보, 힌트, 리더보드. 문제를 관리하는 `/quiz-add` , `/quiz-check` 등 커스텀 명령어.
- **만난 개념:** 상태(state) 관리, 화면 전환, 무작위 섞기(Fisher-Yates), localStorage 리더보드, 커스텀 명령어.
- **교훈:** 반복되는 지시는 명령어로 만들어 두면 매번 똑같이 정확하게 실행된다.

04 Study-04 · API로 AI 서비스 만들기 (냉장고 레시피)

- **만든 것:** 냉장고 사진 → 재료 인식 → 레시피 추천 → 프로필·저장. Streamlit 화면, Google Drive 저장.
- **만난 개념:** API, 엔드포인트, API 키와 환경 변수, Colab 보안 비밀, OpenRouter, 멀티모달, 추론 모델의 `<think>` , 타임아웃, 401/429/5xx 처리, JSON 파일 저장과 원자적 쓰기, PRD(요구사항 문서), **서브에이전트** (code-reviewer 등 3개).
- **교훈:** 외부 서비스는 언제든지 실패할 수 있다고 보고, 실패했을 때 사용자에게 무엇을 해야 하는지 알려 줘야 한다.

05 Study-05 · 교대근무 근무표 + AI 선호도

- **만든 것:** 규칙을 지키며 근무표를 자동으로 짜는 앱에, 사람들의 자유로운 선호("A는 주말 주간 선호")를 AI가 해석해 반영하는 기능.
- **만난 개념:** 하이브리드 설계(AI는 해석, 코드는 계산), 서버와 클라이언트 분리(키는 서버에만), 벌점(penalty) 기반 탐색, 자동 테스트 221개, 서버에이전트 팀 5개와 작업 순서.
- **교훈:** AI에게 모든 걸 맡기지 말고, AI가 잘하는 일과 코드가 잘하는 일을 나눈다.

06 Study-06 · 스킬, MCP, 플러그인 (오늘)

오늘 하루를 순서대로 되짚어 보면, 부품 하나하나를 차례로 만나는 여정이었어요.

1. **스킬 만들기:** `auto-review` . SKILL.md + 점검표 + 보고서 틀. 연습용 예제 파일은 필요 없다고 하셔서 지웠어요.
2. **외부 스킬 받기:** `npx vibeindex add ...` 실패 → 원인은 Node.js가 아니라 **네트워크 허용 목록(403)** → git clone으로 받아 내용을 확인하고 복사 → 안전 검사가 한 번 막음 → 허락 후 설치.
3. **네트워크 설정:** 환경의 Network access에 vibeindex.ai 추가 → `/vibeindex` 로 프로젝트 분석과 도구 추천.
4. **쇼핑 리스트 앱:** HTML 파일 하나, localStorage 저장, textContent로 XSS 방지.
5. **스킬 자동 실행:** 앱을 다 만들자 `auto-review` 가 스스로 켜져서 리뷰.
6. **플러그인의 벽:** 클라우드 세션엔 `/plugin` 이 없음 → 플러그인은 부품 꾸러미라는 걸 이해.
7. **MCP 연결:** playwright 플러그인 = MCP 서버 하나 → `.mcp.json` 에 직접 등록. chrome 없음 → chromium 버전 불일치 → `file://` 금지 → localhost 서버로 해결.
8. **스킬 + MCP 조합:** `browser-check` 스킬을 만들고 auto-review와 연결.
9. **자동 테스트:** 추가·체크·삭제·저장·보안·휴대폰 화면까지 28개 모두 통과.
10. **마켓플레이스와 플러그인 설치 범위:** anthropics/skills 등록 + document-skills를 project 범위로 설정.
11. **문서 스킬 활용:** pptx 스킬로 AI 트렌드 1페이지 PPT, pdf 스킬로 바로 이 노트.

오늘의 가장 큰 교훈

도구가 막히면(설치 명령 실패, 플러그인 불가, 네트워크 차단) **그 도구가 속으로 무슨 일을 하는지**를 이해하면 다른 길이 보여요. "플러그인 설치"는 결국 "부품 파일을 제자리에 놓는 일"이었고, "npx 설치"는 결국 "GitHub에서 파일을 받아 복사하는 일"이었어요.

6부. 스스로 점검하기

답을 가리고 먼저 떠올려 보세요. 해설은 문제 뒤에 모아 뒀어요. 틀려도 괜찮아요. 해설에 적힌 장으로 돌아가 다시 읽으면 돼요.

문제

- 로그인한 뒤 다른 페이지로 가도 로그인이 유지되는 건 무엇 덕분일까요? 서버는 원래 기억력이 없다고 했는데요.
- 코드를 고치고 push했는데, Colab에서 열어 보니 화면이 예전 그대로예요. 확인할 순서 두 가지는?
- 쿠키와 localStorage의 가장 큰 차이 하나는?
- `http://localhost:8765` 와 `http://localhost:8000` 의 localStorage는 서로 공유될까요? 왜 그런가요?
- AI를 부르다 401이 떴어요. 403이 떴어요. 429가 떴어요. 각각 무엇부터 확인할까요?
- API 키를 코드에 직접 쓰면 안 되는 이유와, Colab에서 키를 쓰는 올바른 방법은?
- 입력한 글자를 화면에 넣을 때 `innerHTML` 대신 `textContent` 를 쓴 이유는?
- 커밋과 push의 차이는? 클라우드 세션에서 push를 잊으면 무슨 일이 생길까요?
- `npm install` 과 `npx` 의 차이는?
- 패키지 버전을 `@0.0.82` 처럼 고정하면 좋은 점과 나쁜 점은?
- Study-05에서 AI에게 근무표를 통째로 쓰게 하지 않은 이유는?
- 에이전트와 일반 챗봇의 차이를 한 문장으로 말해 보세요.
- 커스텀 명령어와 스킬의 차이 중 가장 중요한 한 가지는?
- 스킬의 `description` 을 잘 써야 하는 이유는?
- 같은 "코드 리뷰"라도 스킬로 만들 때와 서브에이전트로 만들 때 어떻게 다를까요?
- MCP와 스킬은 각각 Claude에게 무엇을 더해 주나요?
- "앞으로 파일을 고칠 때마다 무조건 문법 검사를 돌려 줘"는 어떤 부품으로 만드는 게 맞을까요? 왜요?
- 플러그인과 마켓플레이스의 관계를 비유로 설명해 보세요.
- "install for all collaborators on this repository"를 고르면 설정이 어디에 적히고, 누구에게 적용되나요?
- `.mcp.json` 을 Study-06 폴더 안으로 옮기면 클라우드 세션에서 Playwright가 안 되는 이유는?

해설

- 쿠키와 세션.** 서버가 로그인 성공 때 세션 번호를 쿠키로 맡기고, 브라우저가 매 요청마다 그 쿠키를 자동으로 같이 보내요. 서버는 그 번호로 세션 장부를 찾아 "그분"임을 알아봐요. (1-7)

2. ① Colab에서 `git pull` (또는 새로 clone)로 새 코드를 받았는지, 서버가 필요한 앱이면 다시 켜는지. ② 브라우저 캐시 때문일 수 있으니 강력 새로고침(`Ctrl+Shift+R`). (1-10, 2-5)
3. 쿠키는 **매 요청마다 서버로 자동 전송**되고, `localStorage`는 서버로 가지 않고 브라우저 안에만 있어요. (1-8)
4. 공유되지 않아요. 포트가 달라서 **출처(프로토콜+도메인+포트)**가 다르기 때문이에요. (1-9)
5. 401 → API 키가 있는지, 맞는지. 403 → 권한이나 접속 차단(허용 목록 등). 429 → 너무 자주 요청했으니 잠시 기다렸다가 다시. (1-3)
6. 코드는 GitHub에 올라가 남이 볼 수 있으니까요. Colab에서는 보안 비밀(🔑)에 넣고, 셀에서 환경 변수로 옮겨 코드가 환경 변수를 읽게 해요. 화면·로그에 출력하지 않아요. (1-11)
7. 입력에 HTML 코드가 섞여 있어도 실행되지 않고 글자 그대로 보이게 하려고요(XSS 방지). (1-11)
8. 커밋은 내 작업 공간에 세이프 포인트를 만드는 것, push는 그걸 GitHub에 올리는 것이에요. 컨테이너는 임시라서 push를 잊으면 세션이 끝날 때 변경이 사라지고, Colab에서도 볼 수 없어요. (2-5, 2-6)
9. `npm install` 은 패키지를 폴더(`node_modules`)에 설치해 두는 것, `npm` 는 설치해 두지 않고 한 번 받아서 바로 실행하는 것이에요. (2-4)
10. 좋은 점: 오늘 확인한 그대로 계속 동작해요. 나쁜 점: 새 버전의 개선이나 보안 수정을 자동으로 못 받으니까, 가끔 직접 올려 줘야 해요. (2-4)
11. AI는 긴 근무표에서 규칙(야간 다음 날 주간 금지 등)을 자주 어기고 시간 계산도 틀려서요. 그래서 AI는 자연어 선호를 규칙 형식으로 바꾸는 일만 하고, 계산은 코드가 맡았어요(하이브리드). (3-4)
12. 챗봇은 말만 하고, 에이전트는 도구를 써서 행동하고 결과를 보며 목표를 이룰 때까지 반복해요. (3-5)
13. 명령어는 **사람이 /이름 으로 불러야만** 실행되고, 스킬은 **Claude가 상황을 보고 스스로 꺼내 쓸 수** 있어요. (4-3, 4-4)
14. Claude는 평소에 스킬의 이름과 설명만 알고 있다가 설명과 맞는 상황에서 본문을 펼치기 때문이에요. 설명이 애매하면 필요할 때 안 켜지거나, 엉뚱할 때 켜져요. (4-4)
15. 스킬은 메인 Claude가 매뉴얼을 보며 대화 흐름 안에서 바로 리뷰해요. 서브에이전트는 별도 책상에서 리뷰하고 보고서만 돌려주며, 도구 목록으로 파일 수정을 구조적으로 막을 수 있어요. (4-5)
16. MCP는 새 **도구(손)**를, 스킬은 그 도구를 쓰는 **절차(방법)**를 더해 줘요. (4-6)
17. **훅**이에요. 스킬이나 CLAUDE.md는 Claude의 판단이 들어가지만, 훅은 프로그램이 이벤트마다 무조건 실행해요. "~할 때마다 반드시"는 훅의 뉘앙스예요. (4-7)
18. 마켓플레이스는 도시락 가게(목록), 플러그인은 도시락(부품 꾸러미)이에요. 가게를 더 알수록 고를 도시락이 많아지지만, 담긴 반찬의 종류(스킬·명령어·에이전트·MCP·훅)는 같아요. (4-9)
19. 저장소의 `.claude/settings.json` (project 범위)에 적히고, 커밋되어 **이 저장소를 쓰는 모든 사람에게** 적용돼요. (4-8)
20. Claude Code는 **작업 폴더**의 `.mcp.json` 을 찾는데, 클라우드 세션은 저장소 맨 위에서 시작하기 때문에 하위 폴더 안의 파일은 못 찾아요. (2-1, 4-12)

부록. 용어 사전

한 줄 정의 모음이에요. 괄호 안 숫자는 자세히 설명한 곳이에요.

웹 · 네트워크

서버	요청을 받아 무언가를 내주는 컴퓨터·프로그램. 기계가 아니라 역할 (1-1)
클라이언트	요청을 보내는 쪽. 보통 브라우저 (1-1)
URL	프로토콜·도메인·포트·경로·쿼리로 된 인터넷 주소 (1-2)
프로토콜	대화 약속. http, https, file 등 (1-2)
도메인	사람이 읽기 쉬운 컴퓨터 이름. vibeindex.ai (1-2)
포트	한 컴퓨터 안의 프로그램 창구 번호. 443, 8765 (1-2)
localhost	"바로 이 컴퓨터". 127.0.0.1 (1-2)
포트 포워딩	안쪽 포트를 바깥에서 접속할 수 있게 이어 주는 것. Colab의 serve_kernel_port_as_window (1-2)
HTTP / HTTPS	웹의 요청·응답 규칙 / 그걸 암호화한 것 (1-3, 1-11)
GET / POST	"가져와 줘" / "이걸 받아 처리해 줘" 요청 방식 (1-3)
헤더	요청·응답에 붙는 부가 정보. 키, 쿠키 등 (1-3)
상태 코드	응답 결과 번호. 2xx 성공, 4xx 요청 쪽 문제, 5xx 서버 문제 (1-3)
HTML / CSS / JS	구조 / 모양 / 동작 (1-4)
반응형	화면 크기에 따라 모양이 바뀌는 디자인 (1-4)
DOM	브라우저가 HTML을 나무 모양으로 기억한 것. JS가 이걸 고쳐 화면을 바꿈 (1-5)
이벤트	클릭·입력 같은 "무슨 일이 생겼다" 신호 (1-5)
콘솔	F12 개발자 도구에서 JS 메시지·오류를 보는 곳 (1-5)
JSON	데이터를 사람·컴퓨터 모두 읽는 글자로 적는 형식 (1-6)
쿠키	서버가 브라우저에 맡기는 작은 메모. 매 요청마다 자동 전송 (1-7)
세션	서버 쪽에서 "이 사용자"를 기억해 두는 장부. 쿠키에는 번호만 (1-7)
localStorage	브라우저 안의 앱 데이터 저장소. 서버로 안 감, 출처별로 따로 (1-8)
sessionStorage	탭을 닫으면 사라지는 브라우저 저장소 (1-8)
출처(Origin)	프로토콜+도메인+포트. 같아야 같은 사이트 (1-9)
CORS	다른 출처의 요청을 서버가 허락하는 규칙 (1-9)
캐시	속도를 위해 가까이 둔 복사본. 옛날 것일 수 있음 (1-10)

강력 새로고침	캐시를 무시하고 새로 받기. Ctrl+Shift+R (1-10)
CDN	세계 곳곳에 웹 파일 복사본을 두는 서비스 (1-10)
XSS	입력값이 코드로 실행되는 공격. textContent로 막음 (1-11)
IP 주소	인터넷 컴퓨터의 진짜 번호 (1-12)
DNS	도메인을 IP로 바꿔 주는 전화번호부 (1-12)
프록시	대신 접속해 주는 대리인. 허용 목록으로 거를 수 있음 (1-12)
허용 목록	가도 되는 곳 목록. 없으면 차단 (1-12)

개발 도구

경로	파일 위치를 적은 글자. 절대/상대, . 과 .. (2-1)
작업 디렉터리	프로그램이 지금 일하는 기준 폴더 (2-1)
숨김 파일	점(.)으로 시작하는 이름. 주로 설정 파일 (2-1)
터미널 / 셸	글자로 명령하는 창 / 그 명령을 실행하는 프로그램 (2-2)
런타임	코드를 실제로 돌려 주는 프로그램. Python, Node.js, 브라우저 (2-3)
Node.js	브라우저 밖에서 JS를 돌리는 런타임 (2-3)
라이브러리 / 패키지	남이 만든 코드 묶음 (2-4)
프레임워크	틀이 정해진 큰 라이브러리. 내 코드를 틀에 끼움 (2-4)
pip / npm	파이썬 / JS 패키지 관리자 (2-4)
npx	설치 없이 한 번 받아 바로 실행 (2-4)
의존성	내 코드가 돌아가는 데 필요한 다른 패키지 (2-4)
버전 고정	특정 버전만 쓰게 막아 두기. @0.0.82 (2-4)
Git / GitHub	변경 이력 관리 프로그램 / 그걸 인터넷에 보관하는 사이트 (2-5)
저장소(repo)	Git이 관리하는 프로젝트 폴더 (2-5)
커밋	메시지를 붙인 세이브 포인트 (2-5)
브랜치	본편과 따로 가는 작업 줄기 (2-5)
clone / pull / push	처음 받기 / 새것 받기 / 올리기 (2-5)
머지 / PR	브랜치 합치기 / 합쳐 달라는 요청서 (2-5)
.gitignore	Git이 무시할 파일 목록 (2-5)
로컬 / 클라우드	내 컴퓨터 / 빌려 쓰는 인터넷 너머 컴퓨터 (2-6)
컨테이너	클라우드 안의 독립된 임시 작업실 (2-6)

배포	앱을 남도 접속할 수 있는 곳에 올리기 (2-6)
환경 변수	프로그램 밖에 붙여 두는 이름표 달린 값. 키 보관에 씬 (1-11)
테스트 / 단언	기대대로 동작하는지 확인 / "반드시 이래야 한다"는 문장 (2-7)
엣지 케이스	빈 값·깨진 값 같은 드문 상황 (2-7)
회귀	새 변경 때문에 예전에 되던 게 망가짐 (2-7)
린터	실행 없이 코드를 읽어 문제를 찾는 도구 (2-7)
Playwright	코드로 브라우저를 조종하는 도구 (2-7)
헤드리스	화면 없이 브라우저 돌리기 (2-7)

AI

머신러닝	데이터로 규칙을 스스로 배우게 하는 방식 (3-1)
학습 / 추론	규칙을 익히는 단계 / 익힌 규칙으로 답하는 단계 (3-1)
모델	학습 결과물. 숫자 문치로 된 규칙 (3-1)
LLM	글을 아주 많이 배운 거대 언어 모델 (3-2)
토큰	LLM이 읽고 쓰는 글 조각 단위. 요금·길이 기준 (3-2)
컨텍스트 윈도우	LLM이 한 번에 볼 수 있는 양. 책상 크기 (3-2)
프롬프트	AI에게 주는 지시. 시스템 프롬프트는 바탕 규칙 (3-2)
환각	그렇듯하지만 틀린 답 (3-2)
추론 모델	답 전에 단계별로 생각하는 모델 (3-3)
멀티모달	글·이미지·음성 등을 함께 다룸 (3-3)
API / 엔드포인트	프로그램끼리 대화하는 창구와 약속 / 그 주소 하나 (3-4)
API 키	서비스가 나를 확인하는 비밀 문자열 (1-11, 3-4)
OpenRouter	여러 AI 모델을 한 창구에서 쓰게 하는 중개소 (3-4)
타임아웃	너무 오래 걸리면 포기하는 시간 제한 (3-4)
하이브리드 설계	AI는 해석, 코드는 계산처럼 역할을 나누기 (3-4)
에이전트	도구를 쓰며 생각→행동→관찰을 반복하는 AI (3-5)

Claude Code

Claude Code	코드 작업을 하는 Claude 에이전트. 터미널·앱·웹에서 씬 (4-1)
세션	대화 한 판. 시작할 때 설정을 읽음 (4-1)

클라우드 세션	claude.ai/code에서 컨테이너로 도는 세션 (4-1, 4-12)
CLAUDE.md	세션마다 자동으로 읽히는 프로젝트 규칙 수첩 (4-2)
커스텀 명령어	사람이 /이름으로 부르는 단축 명령 (4-3)
스킬	Claude가 상황을 보고 스스로 꺼내 쓰는 업무 매뉴얼 (4-4)
SKILL.md	스킬의 본체. name·description·allowed-tools + 절차 (4-4)
점진적 공개	제목만 → 필요하면 본문 → 부속 파일 순으로 읽기 (4-4)
allowed-tools	스킬을 쓰는 동안 묻지 않고 써도 되는 도구 (4-4)
서브에이전트	별도 컨텍스트에서 일하고 보고서만 돌려주는 다른 Claude (4-5)
model: inherit	서브에이전트가 부른 쪽과 같은 모델을 쓰라는 설정 (4-5)
MCP	AI 앱과 바깥 도구를 잇는 공통 규격. USB-C (4-6)
MCP 서버	도구를 제공하는 프로그램. @playwright/mcp (4-6)
.mcp.json	컬 MCP 서버를 적는 파일 (4-6)
커넥터	claude.ai에서 연결하는 Notion·GitHub 등. MCP 기반 (4-6)
훅	정해진 이벤트마다 프로그램이 무조건 실행하는 규칙 (4-7)
settings.json	권한·훅·MCP·플러그인 설정 파일 (4-8)
범위(scope)	설정이 누구에게 적용되나. user / project / local / managed (4-8)
플러그인	스킬·명령어·에이전트·MCP·훅을 묶은 꾸러미 (4-9)
마켓플레이스	플러그인 목록(가게). 대개 GitHub 저장소 하나 (4-9)
claude-plugins-official	기본으로 추가되는 Anthropic 공식 마켓플레이스 (4-9)
enabledPlugins	settings.json에서 컬 플러그인 목록 (4-8, 4-9)
extraKnownMarketplaces	settings.json에 추가로 알려 줄 마켓플레이스 목록 (4-8, 4-9)

여기까지 읽었다면 오늘 공부는 충분해요. 근무 무사히 마치세요. 🌙
 다음 실습 후보: 아직 안 만들어 본 **훅** 만들어 보기